



EMI305 · PRODUCCIÓN DE SOFTWARE · SISTEMAS CIBERFÍSICOS

Análisis del modelo C4: Arquitectura de sistemas en 4 niveles

Constructos, notación, metamodelo y semántica de un lenguaje visual para describir arquitectura de software aplicado a un sistema ciberfísico.

Magíster en Ingeniería Informática · UFRO

EMI305 · Producción de Software · 2026



INSTITUCIÓN ACREDITADA
6 AÑOS
EN TODAS LAS ÁREAS
• GESTIÓN INSTITUCIONAL • DOCENCIA DE PREGRADO
• DOCENCIA DE POSTGRADO • INVESTIGACIÓN
• VINCULACIÓN CON EL MEDIO

¿Qué es el modelo C4?

Origen. Creado por Simon Brown entre **2006 y 2011** como respuesta a la dificultad de comunicar arquitectura con UML. Hoy es estándar de facto en la industria y cuenta con herramientas propias (*Structurizr*).

¿Cuál es su propósito? Describir la **arquitectura estática** de un sistema de software mediante un conjunto de **mapas jerárquicos**, cada uno dirigido a una audiencia distinta — «zoom» progresivo desde el contexto de negocio hasta el código.

¿Qué tipo de sistemas permite representar? Cualquier sistema de software compuesto por **unidades desplegables que se comunican**: web, microservicios, móviles, integraciones y también **sistemas ciberfísicos**, donde los nodos físicos se modelan como contenedores.

¿En qué contexto se utiliza? Documentación de arquitectura, *onboarding* de equipos, revisiones de diseño y comunicación con *stakeholders* no técnicos. Su premisa: la arquitectura no es una vista única — el contexto le importa al negocio, los contenedores a operaciones, los componentes al desarrollador.

LOS 4 NIVELES

Level 1: Context

Sistema + usuarios + sistemas externos

Level 2: Container

Aplicaciones, bases de datos, nodos físicos

Level 3: Component

Módulos funcionales dentro de un container

Level 4: Code

Clases, funciones, métodos

Constructos y conceptos del modelo C4

CONSTRUCTO	ROL	NIVEL
Person	Usuario o actor humano	1
SoftwareSystem	El sistema descrito, o uno externo con el que interactúa	1
Container	Unidad desplegable y ejecutable: app, BD, cola, nodo físico	2
Component	Agrupación lógica dentro de un contenedor	3
Code element	Clase, interfaz, función. C4 no define notación propia: se usa UML	4
Relationship	Interacción dirigida, con <i>description</i> y <i>technology</i>	todos

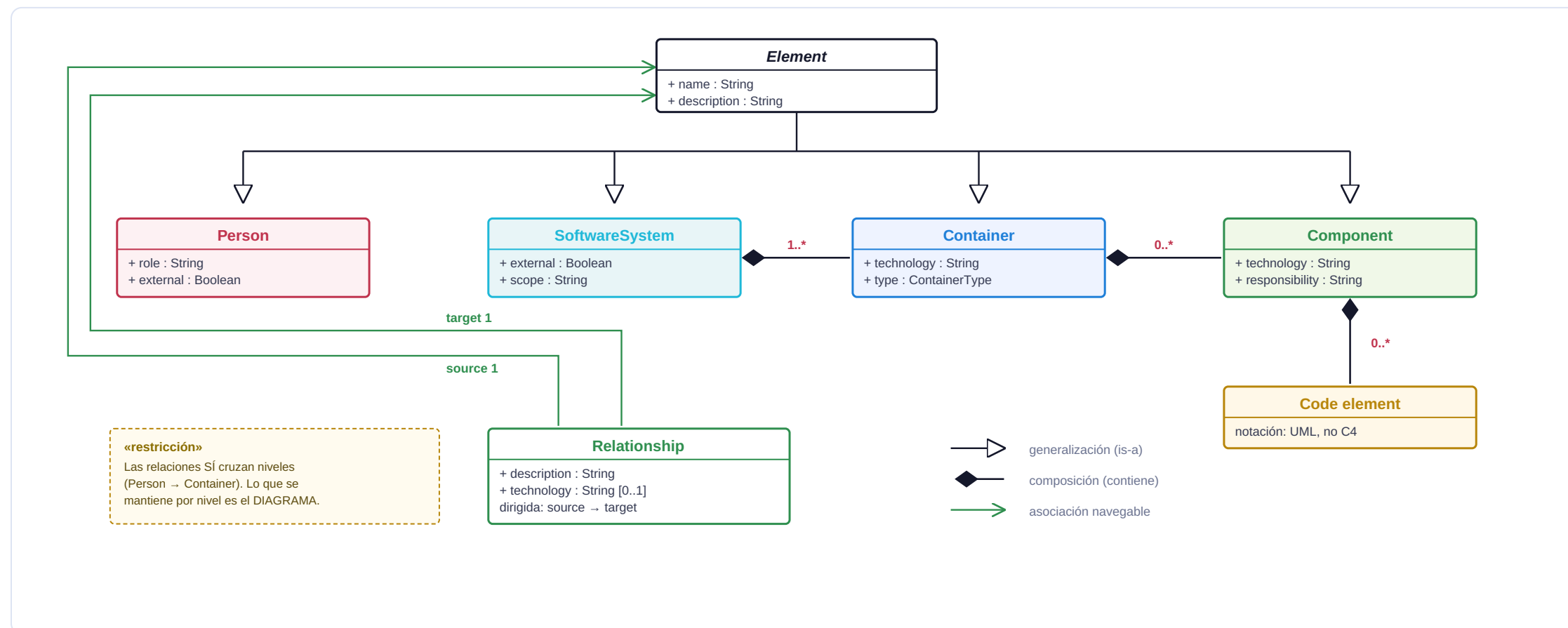
Cómo se relacionan. Los cuatro estructurales forman una jerarquía de **composición**: SoftwareSystem $\blacklozenge \rightarrow$ Container $\blacklozenge \rightarrow$ Component $\blacklozenge \rightarrow$ Code element. **Relationship** es transversal: conecta dos *Element* cualesquiera, **incluso de niveles distintos** (una Person usa un Container).
C4 añade además vistas suplementarias —*System Landscape*, *Dynamic* y *Deployment*— con sus propios elementos (`deploymentNode`).

RESTRICCIONES (REGLAS DE BUENA FORMACIÓN)

- **La regla central es de alcance, no de conexión:** cada diagrama muestra **un solo nivel de zoom**. Poner componentes en un diagrama de contexto, o clases en uno de contenedores, es el error más común.
- Las **relaciones sí cruzan niveles**: una *Person* se conecta con un *Container*, y en un diagrama de contenedores aparecen las personas y sistemas externos que tocan esos contenedores.
- Un **SoftwareSystem** externo se modela como **caja negra**: no se abre aunque sepamos qué contiene.
- Toda Relationship es **dirigida** y debe llevar *description*; declarar la *technology* es opcional pero recomendado.
- Todo elemento tiene **name** y **description**.

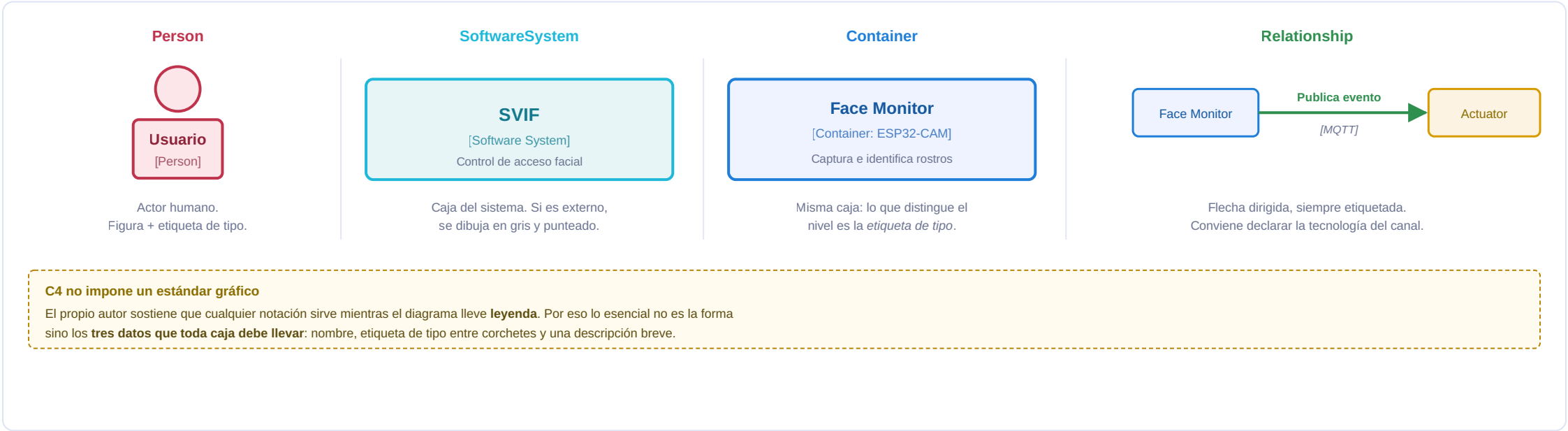
Nada de esto lo **impone** una gramática: son convenciones documentadas. Solo al escribir el modelo en *Structurizr DSL* aparecen restricciones verificables por herramienta.

Relaciones entre constructos (diagrama de clases simplificado)



Metamodelo simplificado propuesto a partir de la especificación del C4 model (Brown, 2016) y de la estructura de datos del Structurizr DSL.

Notación visual estándar de C4



¿Qué significa cada nivel? (Transformación de abstracción)

1 Context

¿Quiénes son los usuarios? ¿Qué otros sistemas existen? ¿Límites del sistema?

2 Container

¿Qué aplicaciones/nodos/DBs hay? ¿Cómo se despliegan? ¿Tecnología base?

3 Component

¿Qué módulos funcionales hay dentro? ¿Cómo se comunican? ¿Responsabilidades?

4 Code

¿Qué clases/métodos implementan? ¿Patrones de diseño? ¿Detalles técnicos?

EN SVIF:

Context: Usuario quiere acceder, cámara detecta, cerradura se abre.

Container: Dos nodos ESP32 (Face Monitor + Access Actuator) comunicados por MQTT.

Component: Dentro de Face Monitor: captura, detección, identificación, MQTT sender.

Code: `monitorearPresenciaTask()`, `capturarImagen()`, `identificarRostro()`, `publicIdentificationEvent()`.

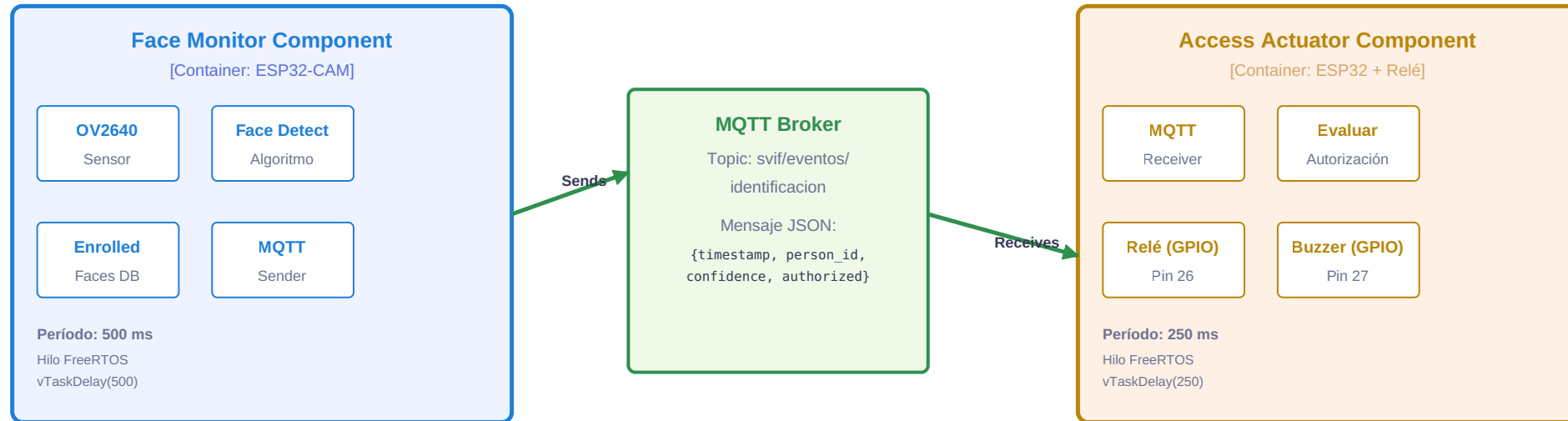
Context: Sistema en contexto



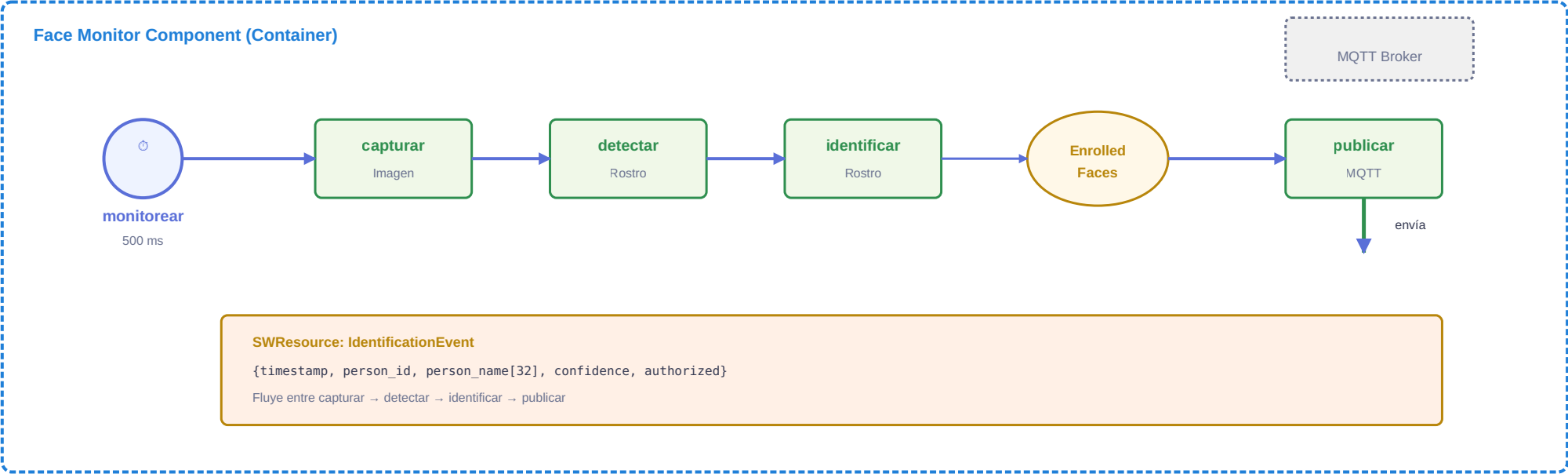
Regla del nivel 1: el sistema no se abre.

Aquí no aparecen ni los dos nodos ESP32 ni el broker MQTT — son *contenedores*, y descomponer el sistema en este nivel sería mezclar niveles de abstracción, que es justamente lo que las reglas de buena formación prohíben.

Container: Nodos y tecnología



LEVEL 3 **Component: Dentro de Face Monitor**



¿Por qué C4 es útil y cuáles son sus limitaciones para CPS?

FORTALEZAS DE C4

- **Escalable.** 4 niveles claros permiten comunicar desde CEO hasta desarrollador sin abrumar.
- **Agnóstico.** No asume tecnología; funciona igual para Java, Node, Go, IoT.
- **Composable.** Se puede expandir con diagramas adicionales (deployment, infraestructura).
- **Bidireccional.** El código puede generar C4 (inversión) o C4 guiar el código (forward-engineering).
- **Comunicación.** Diagramas visuales ayudan a stakeholders no técnicos a entender estructura.

LIMITACIONES PARA CPS

- **No muestra tiempo.** ¿Cada cuánto se ejecuta monitorear? (500 ms en SVIF) → necesita anotación.
- **No muestra transiciones.** ¿Qué ocurre si authorized=true vs false? → necesita lógica de control.
- **No muestra hardware.** ¿Qué pin? ¿Qué sensor? → apenas menciona HWResource.
- **No muestra restricciones temporales duras.** ¿Deadline? ¿Jitter tolerable? → requiere DSL más específico.
- **Abstracción variable.** Un "Container" puede ser app web o nodo ESP32 con 2KB RAM → demasiado genérico.

¿QUÉ ASPECTOS PODRÍAN MEJORARSE? · ¿ES ADECUADO PARA MI DOMINIO?

Mejoras que propondría. (i) Un atributo estándar de **periodicidad** en Container/Component, hoy relegado a la descripción libre. (ii) **Estereotipos de hardware** para distinguir un nodo embebido de un servicio en la nube. (iii) Un **enlace explícito a requisitos** o a objetivos, que permita justificar por qué existe cada contenedor. (iv) Reglas de validación formales: hoy la «buena formación» es convención documentada, no un metamodelo verificable —salvo que se use Structurizr DSL.

¿Adecuado para CPS? Sí, pero parcialmente. El nivel 2 describe SVIF con precisión —dos nodos y un canal MQTT— y es la vista que efectivamente usaría para explicar el sistema. Lo que no alcanza a expresar son los **500 ms** de muestreo, el pin del relé y la bifurcación *desbloquear v alarmar*. Esa carencia es justamente lo que motiva un **DSL propio para CPS**: C4 aporta contexto y contenedores; el DSL aporta comportamiento y tiempo. **Son complementarios, no competidores.**

REFERENCIAS

Fuentes consultadas

Brown, S. (2016). *Software architecture for developers. Volume 2: Visualise, document and explore your software architecture*. Leanpub. Especificación en línea: <https://c4model.com>
Language and tool for visualizing software architecture at four levels of abstraction.

Fowler, M. (2010). *Domain-Specific Languages*. Addison-Wesley.
Comprehensive guide to designing and implementing DSLs; discusses C4 within broader modeling context.

Navarro, C., Devia, L., Labra Gayo, J. E., & Cares, C. (2025). An agent-oriented model-driven development process for cyber-physical systems. *CibSE 2025* (pp. 150–164). SBC.
Process combining iStar 2.0 (CIM), DSL PIM, and C++ PSM; includes C4 for Level 2 container view.

UML 2.5.1 Specification. (2017). Object Management Group (OMG). <https://www.omg.org/spec/UML/>
Standard for unified modeling language; C4 is positioned as simpler alternative for architecture views.